LOGO Video Text Spotting

Technical documentation: architecture, methods, models and internal structure

A guide for first-time readers of the *Logo_Video_Text_Spotting* repository

What	Summary
Project	B.Tech major project: re-implementation of **LOGO** ("Video Text Spotting with Language Collaboration and Glyph Perception Model", arXiv 2405.19194) plus two extensions, **C2** soft language re-scoring and **C3** track-level text voting.
Task	Find every piece of text in every frame of a video, read it, and follow each text instance over time with a stable track ID.
Code base	Python package `logo` (44 modules, about 4,802 lines), 35 command-line tools, 3 pipeline scripts, 52 test files (250 tests, all passing on 2 November 2026).
Hardware target	One NVIDIA GeForce RTX 3050 6 GB laptop GPU; every design choice respects that budget.
Datasets	ICDAR2013 Video, ICDAR2015 Video, DSText.
Headline result	ICDAR2015 Video (val): tracking MOTA 45.80, IDF1 53.06; spotting IDF1 32.15. ICDAR2013 Video (val): tracking MOTA 66.18, IDF1 61.54; spotting IDF1 49.57.

Document generated on 2 November 2026 from the working tree on top of commit `ba98cef`. Every number in it was read from files in the repository (`runs/*/metrics.json`, `docs/`, `configs/`).

Contents

1 Introduction . . . 4
1.1 The problem: video text spotting . . . 4
1.2 What LOGO adds . . . 4
1.3 What this repository contributes . . . 4
2 Vocabulary . . . 4
3 System architecture . . . 5
3.1 The pipeline at a glance . . . 5
3.2 Runtime environments and third-party code . . . 6
3.3 Engineering conventions . . . 6
3.4 Package structure . . . 7
4 Repository layout . . . 8
5 Data layer . . . 8
5.1 Datasets . . . 8
5.2 From raw files to the GT contract . . . 9
5.3 Frames and splits . . . 10
6 Stage 1 — Text detection . . . 10
6.1 Plan A: PP-YOLOE-R-s (PaddlePaddle) . . . 10
6.2 Plan B: YOLO11s-OBB (Ultralytics) — used . . . 11
7 Stage 2 — Language Synergy Classifier (LSC) . . . 11
7.1 Idea . . . 11
7.2 Rectifying a box (logo.lsc.roi_rotate) . . . 11
7.3 The ABINet model . . . 12
7.4 The LSC stage (logo.lsc.run_lsc) . . . 12
7.5 Fine-tuning the LSC (logo.lsc.train_lsc) . . . 12
8 Glyph supervision (GS) . . . 13
9 Stage 3 — Score fusion and the C2 extension . . . 13
10 Visual-Position Mixed Module (VPMM) . . . 14
11 Stage 4 — Tracking . . . 15
11.1 The IoU tracker (tracker.type: simple) . . . 15
11.2 The embedding tracker (tracker.type: embed) . . . 15
11.3 The LST-Matcher tracker (tracker.type: lst) . . . 15
12 Stage 5 — Track-level text voting (C3 extension) . . . 17
13 Stage 6 — Evaluation . . . 17
13.1 Matching rules (logo.eval.match) . . . 17
13.2 Metrics . . . 17

14 File formats and run layout . . . 18
14.1 Stage files . . . 18
14.2 Anatomy of a run folder . . . 18
15 Configuration system . . . 18
15.1 Experiment family . . . 19
16 Running the project . . . 19
16.1 Setup . . . 19
16.2 End-to-end recipe . . . 20
16.3 Demo . . . 20
17 Tool catalogue . . . 21
18 Experiments and results . . . 22
18.1 Ablation . . . 22
18.2 Final held-out evaluation . . . 22
18.3 Efficiency (batch 1, RTX 3050 6 GB, 200 frames) . . . 22
18.4 Error analysis . . . 23
19 Design decisions, deviations and limitations . . . 23
19.1 Decisions and deviations from the paper . . . 23
19.2 Known limitations and open items . . . 24
20 Testing . . . 24
Appendix A Model inventory . . . 25
Appendix B References . . . 25

1 Introduction

1.1 The problem: video text spotting

A **video text spotter** answers three questions for every frame of a video: *where* is text (detection), *what* does it say (recognition), and *which* earlier text instance is it (tracking). The output is a list of boxes, each with a frame number, a track ID that stays the same while the same physical text is visible, a four-corner box (quadrilateral, “quad”), a confidence score and the recognised string.

Text in video is hard: it is small, blurred by motion, seen at an angle and often only partly visible. Many detector mistakes are text-like patterns (fences, windows, stripes) that a recogniser cannot read. LOGO exploits that observation.

1.2 What LOGO adds

- **Language Synergy Classifier (LSC).** A text recogniser (ABINet) reads every detected box. If it cannot read anything, the box is probably not text. That language signal is fused with the detector score to re-score each box.
- **Glyph Supervision (GS).** While training the recogniser, an extra small decoder must predict a pixel mask of the characters (glyphs). This auxiliary task makes the recogniser’s features more robust to blur and low resolution. The head is dropped at inference time.
- **Visual-Position Mixed Module (VPMM).** The recogniser’s visual features are combined with an embedding of the text’s centerline position into one vector per box, used for tracking.
- **LST-Matcher.** The association module from GoMatching links boxes across frames with a short-term matcher (previous frame) and a long-term matcher (last six frames).

1.3 What this repository contributes

- A modular re-implementation of the full LOGO pipeline in which every stage is a separate command that reads and writes files with a fixed format (the **file contract**).
- Two research extensions: **C2** soft fusion, which uses the recogniser’s probability instead of a 0/1 decision, and **C3** track-level voting, which gives every track a single text and removes weak tracks.
- A complete evaluation, ablation, statistics and paper-figure tool chain, plus a demo that turns any mp4 into an annotated video.

> **How to read this document.** Sections 2–4 give the big picture (vocabulary, architecture, folders). Sections 5–13 walk through the pipeline in data-flow order, one stage per section, covering the method, the model and the code that implements it. Sections 14–17 cover file formats, configuration, how to run things and the tool set. Sections 18–20 report results, decisions, limitations and tests.

2 Vocabulary

The code and this document use these terms throughout.

Term	Meaning
quad	Four corner points (8 numbers x1 y1 ... x4 y4, pixels, y pointing down) of a possibly rotated text box. Canonical order is clockwise in image coordinates (`logo.utils.geometry.order_quad`).
dont_care	Ground-truth (GT) box whose text is unreadable (e.g. `###`, `##DONT#CARE##`). It is neither rewarded nor penalised during evaluation.
stage	One step of the pipeline: det, lsc, fuse, track, vote, eval.
exp_id / experiment	Name of a run. All outputs go to `runs/<exp_id>/`; the config file sets it.
det_exp / lsc_exp	Experiment whose det or lsc outputs another experiment re-uses (so expensive stages are not re-run).
LSC	Language Synergy Classifier: ABINet recogniser used to decide whether a box contains text.

Term	Meaning
lang_hard / lang_soft	Language score of a box: 0/1 (anything read?) or a probability in [0, 1] (C2).
score_fused	(1 − w) · detector score + w · language score.
GS	Glyph Supervision: auxiliary glyph-mask segmentation loss during LSC training.
VPMM	Visual-Position Mixed Module: per-box tracking feature from recogniser features + centerline position.
LST / LST-Matcher	Long- and Short-Term Matcher (from GoMatching): Transformer-based association of boxes over time.
track / trajectory	All boxes that share one track_id within a video.
MOTA, MOTP, IDF1	Standard multi-object tracking metrics (Section 13). Higher is better.
tracking vs spotting	Tracking metrics ignore text; spotting metrics also require the recognised text to equal the GT text.
rule 5	Project rule: the test split may only be read when a command is given `--final`.
Plan A / Plan B	The two detector options: PP-YOLOE-R in PaddlePaddle (A) and YOLO11s-OBB in Ultralytics (B).

3 System architecture

3.1 The pipeline at a glance

The system is a linear chain of six stages. A video is first decoded into JPEG frames. The detector proposes rotated boxes; the LSC crops and reads each box; fusion re-scores the boxes; the tracker links boxes into trajectories; voting settles one text per trajectory; evaluation compares the result with ground truth. Figure 1 shows the chain and the file each stage writes.

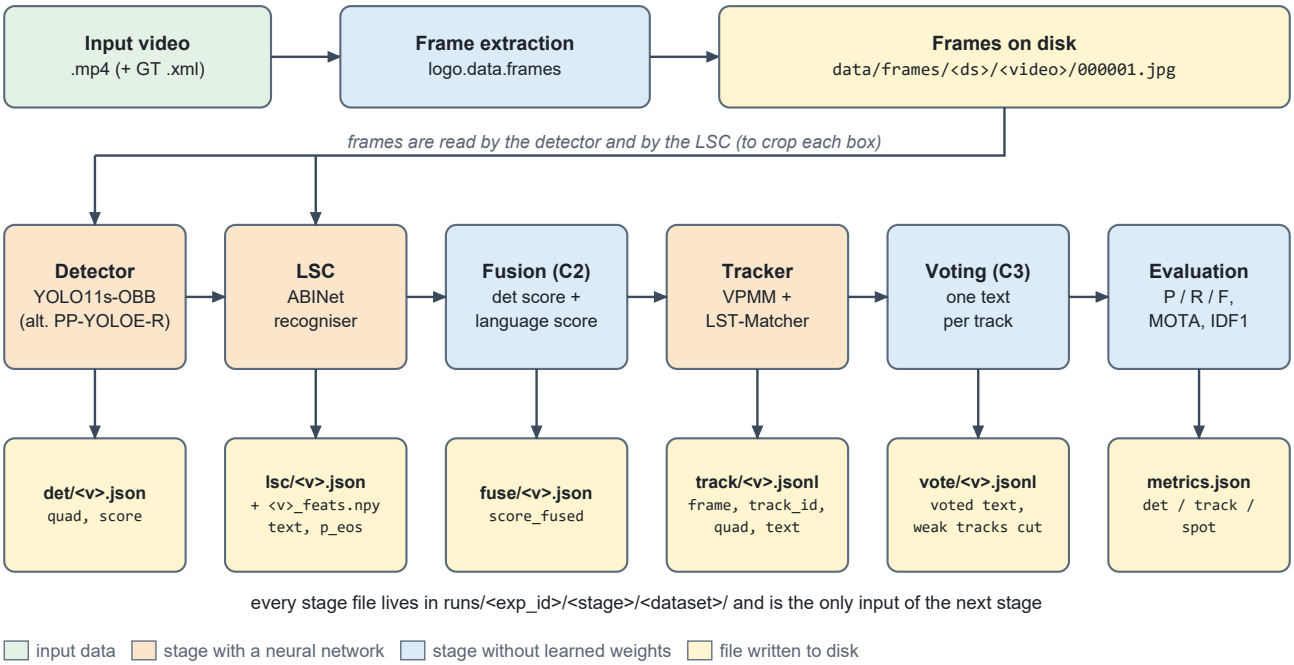


Figure 1. End-to-end pipeline. Top: frame extraction. Middle: the six stages. Bottom: the file each stage writes. Each stage reads only the file of the stage before it (the evaluator also reads the GT json in data/processed).

Each stage is an independent Python module with a command-line interface (`python -m logo.<package>.<module> --config <yaml>`). `scripts/run_pipeline.py` runs any contiguous range of stages for one config, skipping stages whose outputs already exist. The design has four practical benefits:

- **Resumability.** A crash costs only the current stage; finished files are reused.
- **Re-use across experiments.** An ablation that only changes the fusion weight re-runs fusion, tracking and evaluation, not the costly detector or recogniser (`pipeline.det_exp` / `pipeline.lsc_exp` in the config).
- **Two Python environments.** The optional PaddlePaddle detector runs in its own environment; the rest of the pipeline only sees its json output.

- **Debuggability.** Every intermediate result is a human-readable json/jsonl file that the visualisation tools can draw.

3.2 Runtime environments and third-party code

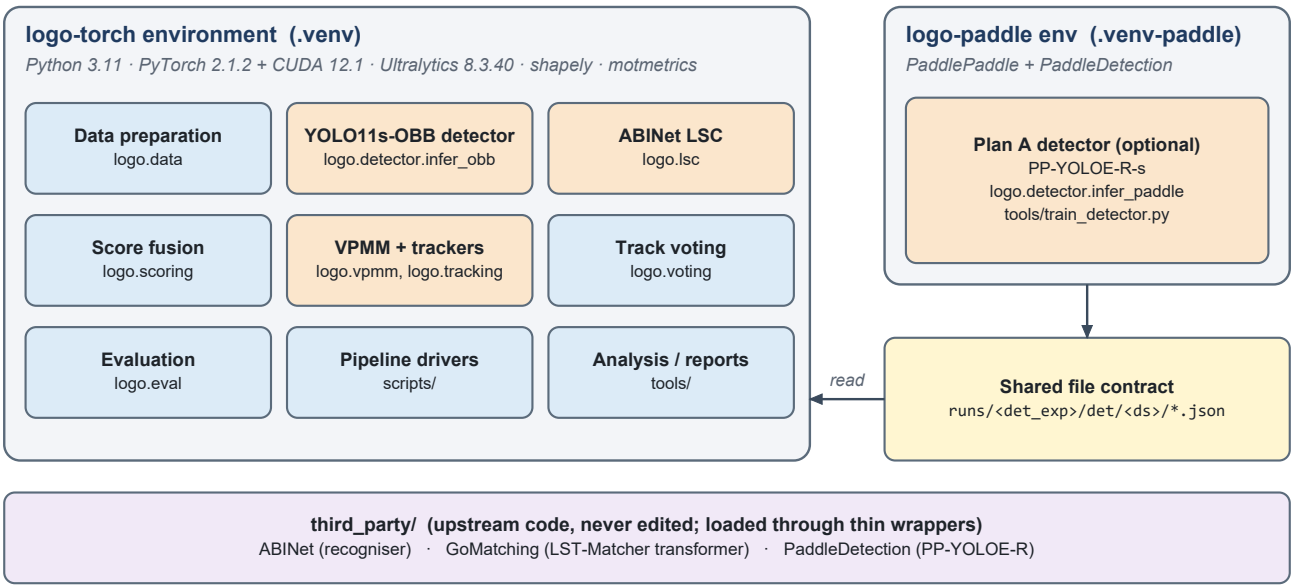


Figure 2. Runtime view. Almost everything runs in the PyTorch environment. The Paddle environment only hosts the optional Plan A detector and talks to the rest through files. Upstream repositories in third_party/ are imported unchanged through thin wrappers.

Three upstream projects live in `third_party/` and are never edited:

- `third_party/ABINet` — the recogniser. `logo/lsc/abinet_wrapper.py` is the only place that adds it to `sys.path`. ABINet star-imports the old fastai 1.0.60, which does not work with PyTorch 2; the wrapper registers a tiny in-memory `fastai.vision` module that supplies the few names ABINet uses.
- `third_party/GoMatching` — only its pure-PyTorch `Transformer` class is loaded, by file path, in `logo/tracking/lst_adapter.py`. GoMatching itself depends on detectron2, which this project avoids.
- `third_party/PaddleDetection` — used by the Plan A detector (training and inference) in the Paddle environment.

3.3 Engineering conventions

The same conventions apply in every module. Knowing them makes the code easy to follow.

Convention	What it means in the code
Fixed file contract	Each stage writes a documented json/jsonl schema (Section 14). Module docstrings state exactly which files are read and written.
Atomic writes	`save_json` / `save_jsonl` / `save_cache` write to a `.tmp` file and rename it, so a crash never leaves a half-written file that would later be mistaken for a finished one.
Skip if done	Stages skip videos whose output exists unless `--overwrite` / `--force` is given.
Run directory	`runs/<exp_id>/` holds `config.yaml` (snapshot of the config used), `log.txt` (appended) and `metrics.json` (results; extra info under `meta.<key>`).
Seeding	`logo.utils.io.set_seed` seeds Python, NumPy and PyTorch from the config `seed` (42).
Rule 5: test split guard	`check_split('test')` raises unless the caller passes `--final`, so the held-out split cannot be used for tuning by accident.
1-based frames	Frame files are `000001.jpg`, ... and json frame keys are strings `"1"`, `"2"`, ... matching the GT XML.
Project root	`$LOGO_ROOT` if set, else the repository root. The demo uses this to run the real stages inside a temporary folder.
Upstream untouched	Third-party code is wrapped, never patched. Deviations are logged in `docs/decisions.md`.

3.4 Package structure

The Python package `logo` lives in `src/logo/` and is installed in editable mode. Figure 3 shows its layers: higher layers import lower ones, never the other way round.

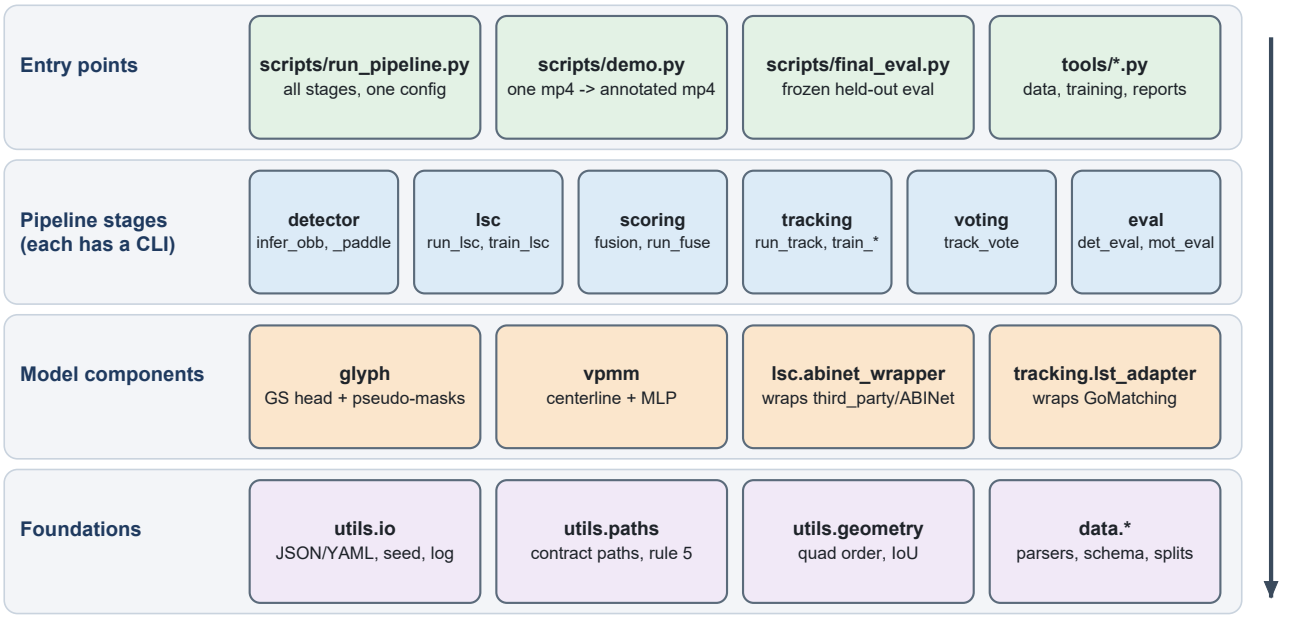


Figure 3. Layered package structure. Entry points orchestrate stages; stages use model components; everything relies on the shared foundations.

Module	Responsibility	Key functions / classes
`utils/io.py`	JSON/YAML I/O, seeding, logging	`load_json, save_json, load_yaml, set_seed, get_logger`
`utils/paths.py`	Contract paths, dataset/split checks	`project_root, check_split, frame_path, gt_path, stage_dir`
`utils/geometry.py`	Quad ordering and polygon IoU	`order_quad, signed_area, quad_iou`
`data/parsers.py`	Raw GT XML -> GT objects	`parse_frames_xml, is_dont_care, find_annotations`
`data/convert_gt.py`	Build a GT json per video	`build_gt, convert_video, probe_video`
`data/frames.py`	Video -> JPEG frames	`extract_frames`
`data/splits.py, schema.py`	Train/val/test lists; GT validation	`make_splits, validate_gt, list_videos`
`detector/infer_obb.py`	Plan B YOLO-OBB inference	`UltralyticsOBB, results_to_rows`
`detector/infer_paddle.py`	Plan A PP-YOLOE-R inference + shared frame loop	`PaddleDetector, run, detect_video`
`detector/prepare_obb_data.py`	GT -> Ultralytics OBB training set	`prepare, quad_to_label`
`lsc/roi_rotate.py`	Rectify a quad into a 32 x 128 crop	`crop_quad, order_vertices`
`lsc/abinet_wrapper.py`	Load and run ABINet	`ABINetRecognizer, decode`
`lsc/run_lsc.py`	LSC stage (det -> lsc + feats)	`run_video, pick_ckpt`
`lsc/dataset.py, train_lsc.py`	LSC crops dataset, augmentations, fine-tuning	`LSCDataset, augment, train`
`glyph/branch.py`	Glyph segmentation head and loss	`GlyphHead, BackboneTap, seg_loss`
`glyph/pseudo_labels.py, dataset.py`	k-means glyph pseudo-masks; dataset with masks	`kmeans_mask, GlyphLSCDataset`
`scoring/fusion.py, run_fuse.py`	Score fusion (hard / soft) and fuse stage	`fuse, lang_score, fuse_video`
`vpmm/geometry.py, module.py`	Centerline and VPMM network	`centerline, VPMM`

Module	Responsibility	Key functions / classes
`tracking/simple_tracker.py`	IoU (ByteTrack-style) and embedding trackers	`run_tracker, match, quad_iou_matrix`
`tracking/lst_adapter.py`	LST-Matcher on plain tensors	`LSTAdapter.associate, .losses`
`tracking/run_track.py`	Track stage, all tracker types	`run, run_lst, filter_trajectories`
`tracking/cache_dataset.py, train_lst.py, train_embed.py`	Tracker training data and training loops	`LSTCacheDataset, train`
`voting/track_vote.py`	C3 track-level text voting	`vote_tracks, attach_scores`
`eval/match.py, det_eval.py, mot_eval.py`	Matching, detection and MOT metrics	`prepare_frame, evaluate_detection, evaluate_mot`

4 Repository layout

Path	Contents
`src/logo/`	The Python package (Section 3.4).
`scripts/`	`run_pipeline.py` (stage driver), `demo.py` (video demo, optional Gradio page), `final_eval.py` (held-out evaluation), plus three small data-prep wrappers.
`tools/`	35 utilities: data audit and conversion, detector/LSC/tracker data builders, sweeps, ablation, statistics, error analysis, table and figure generation (Section 16).
`configs/`	YAML experiment configs. `base.yaml` lists every key; `FINAL.yaml` is the frozen final system.
`splits/`	`<ds>_{train,val,test}.txt`: one video ID per line.
`data/`	`raw/` (original videos + XML), `frames/`, `processed/` (GT json), `obb/` and `det_dota/` (detector training sets), `lsc/` (recogniser crops), `cache/` (tracker training cache). Not in git.
`runs/`	One folder per experiment (Section 14.2). Not in git.
`models/`	Pre-trained weights (YOLO11s-OBB, PP-YOLOE-R, ...).
`third_party/`	ABINet, GoMatching, PaddleDetection (upstream, unmodified).
`docs/`	Data report, baseline report, ablation, decision log, generated tables `docs/tables/` and figures `docs/figs/`.
`paper/`	LaTeX tables and vector PDF figures generated from run outputs.
`tests/`	52 pytest files (250 tests).
`ARCHITECTURE.md`	Design for a planned web front-end/back-end (FastAPI + React). **Not implemented** in the current code.
`PROJECT_STATE.md`	Early progress log (stops at task P01; the git history and docs/ are newer).

5 Data layer

5.1 Datasets

Three public benchmarks are supported. All share the same XML annotation layout, so a single parser handles them. Only the official *train* videos have box annotations for ICDAR2015 Video and DSText; their test annotations are not publicly available.

Dataset	Raw videos (train / test)	Test GT boxes	Notes
ICDAR2013 Video (`icdar2013v`)	13 / 15	present	ASCII XML; zig-zag point order; non-ASCII letters stored as '?'.
ICDAR2015 Video (`icdar2015v`)	25 / 24	not available	UTF-8 XML; main benchmark of this project.
DSText (`dstext`)	50 / 50	not available	Dense small text; videos grouped by scenario (Activity, Driving, Game, Sports, Street View ...).

Statistics after splitting (`tools/dataset_stats.py`; text height = shorter side of the minimum-area rectangle of each readable box):

Dataset	Split	Videos	Frames	Boxes	Tracks	dont_care %	Median height px	Height < 20 px %
icdar2013v	train	10	4,068	18,042	600	16.81	30.5	29.38
icdar2013v	val	3	1,419	8,092	350	0.0	26.1	37.68
icdar2013v	all	13	5,487	26,134	950	11.61	29.0	32.29
icdar2015v	train	20	10,320	62,140	2,731	39.26	23.9	37.01
icdar2015v	val	5	3,130	9,654	293	25.27	40.8	11.01
icdar2015v	all	25	13,450	71,794	3,024	37.38	26.4	32.84
dstext	train	39	9,540	674,108	13,432	17.52	18.2	54.99
dstext	val	11	2,563	179,596	3,193	15.89	19.0	53.54
dstext	all	50	12,103	853,704	16,625	17.18	18.4	54.68

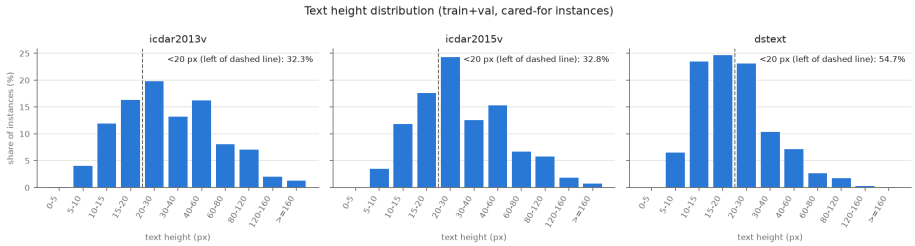


Figure 4. Distribution of text heights per dataset (docs/figs/text_height_hist.png). DSText is dominated by text smaller than 20 px, which is why the detector input is kept at 1024 x 576.

5.2 From raw files to the GT contract

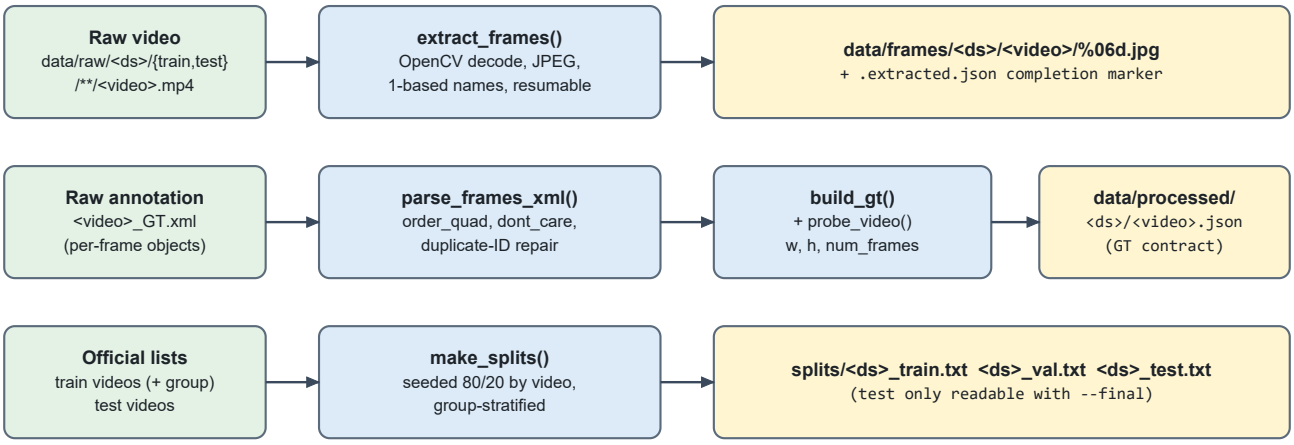


Figure 5. Data preparation. Frames, GT json and split lists are produced once and then read by every later stage.

`logo.data.parsers.parse_frames_xml` reads `<Frames><frame ID><object ID Transcription ...><Point x y/> x4`. For each object it:

- re-orders the four points with `order_quad`: valid quads keep the annotator's first point and are reversed only if counter-clockwise; self-intersecting "bow-tie" quads (common in ICDAR XML) are re-sorted by angle around the centroid;
- marks it `dont_care` if the text is empty or one of `##DONT#CARE##`, `#DON'T CARE#`, `###`, `#UNREADABLE#`;
- normalises the `Quality` attribute (fixing the typo `MODERTE`);
- drops exact duplicate objects and gives a fresh ID to a different box that re-uses an ID in the same frame, so track IDs stay unique per frame.

`convert_gt.build_gt` then writes one json per video in which *every* frame 1..N has a key (empty list if nothing is annotated):

```
{"video": "Video_2_1_2", "dataset": "icdar2015v", "width": 720, "height": 480, "num_frames": 450,
 "frames": {"1": [{"track_id": 3, "quad": [x1,y1,...,x4,y4], "text": "EXIT",
                   "dont_care": false, "quality": "MODERATE"}, ...], "2": [...], ...}}
```

5.3 Frames and splits

- **Frames.** `extract_frames` decodes with OpenCV and writes `data/frames/<ds>/<video>/%06d.jpg`, named by the original 1-based frame index (also when a stride skips frames). A marker file `.extracted.json` is written last, so its presence means “complete” and re-runs skip the video.
- **Splits.** The official train videos are split 80/20 *by video* (never by frame, so no video leaks between train and val), seeded with 42 and stratified by group (the DSText scenario folder). The official test videos form the test list. Result: icdar2013v 10/3, icdar2015v 20/5, dstext 39/11 train/val videos. `check_disjoint` asserts that no video appears twice.
- **Held-out evaluation.** Because test labels are missing for ICDAR2015 Video and DSText, the decision log (`docs/decisions.md`) records *val* as the held-out split for the final evaluation. The same is recorded for ICDAR2013 Video (see Section 19.2 for an open question about that entry).

6 Stage 1 — Text detection

The detector finds rotated text boxes in each frame. It is an **oriented bounding box (OBB)** detector: every box is a rotated rectangle given by four corners, which suits slanted text better than axis-aligned boxes. Two detector plans exist; both write the same output:

```
runs/<det_exp>/det/<ds>/<video>.json
{"video": ..., "dataset": ..., "frames": {"1": [{"quad": [8 floats], "score": 0.93}, ...], ...}}
```

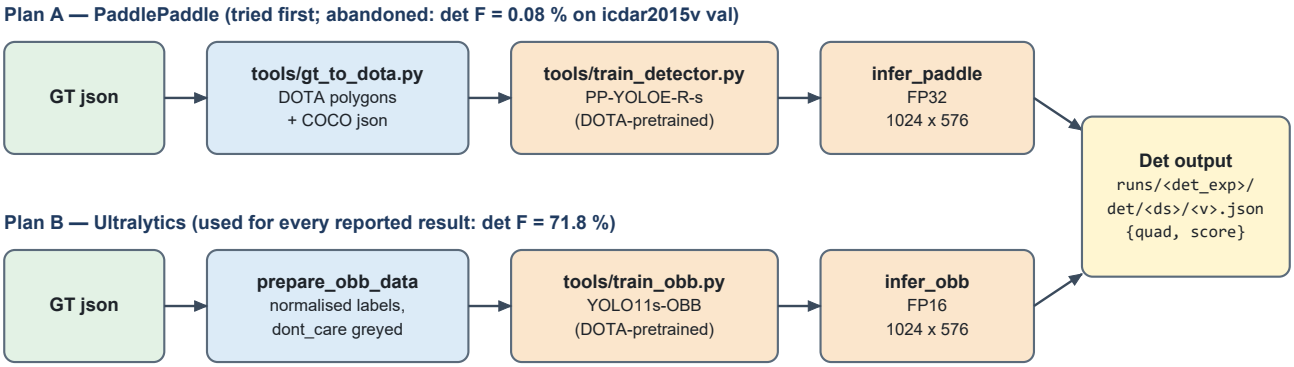


Figure 6. The two detector plans. Plan B is the detector behind every reported number.

6.1 Plan A: PP-YOLOE-R-s (PaddlePaddle)

PP-YOLOE-R is PaddleDetection’s anchor-free rotated detector (small variant, CRN backbone, pre-trained on the DOTA aerial-image dataset). `tools/gt_to_dota.py` converts GT into DOTA polygon files plus COCO json with a single class `text` (dont_care boxes are omitted, not marked “difficult”, because PaddleDetection would still train on them). `tools/train_detector.py` runs PaddleDetection’s trainer with seeding, AMP and auto-resume; `logo.detector.infer_paddle` runs inference in FP32 at 1024 x 576.

> **Outcome.** The fine-tuned PP-YOLOE-R detector did not work on this data: on icdar2015v val it reached 4 true positives against 2,986 false positives (F = 0.08 %, `docs/baseline_report.md`). The project therefore switched to Plan B. Experiments named without the `_yolo` suffix (`E4-base`, `E4-lsc`, `E5-lst`) used Plan A boxes and have near-zero scores.

6.2 Plan B: YOLO11s-OBB (Ultralytics) — used

YOLO11s-OBB is the small oriented-box variant of Ultralytics YOLO11 (9.7 M parameters), also DOTA-pretrained. It runs in the main PyTorch environment.

- **Training data** (`logo.detector.prepare_obb_data`): every 5th train frame and every 10th val frame; one label line `0 x1 y1 ... x4 y4` per readable box, coordinates normalised to [0, 1]; images are hard links to the frames, except that dont_care regions are painted grey (value 114) so that unreadable text is taught neither as text nor as background.
- **Training** (`tools/train_obb.py`, config `det_yolo11s_obb_text.yaml`): 50 epochs, image size 1024, batch 4, AMP, early-stopping patience 15, mosaic off for the last 5 epochs, horizontal flip 0.5, no rotation augmentation, 0 data-loader workers (the host has 7.6 GB RAM).
- **Inference** (`logo.detector.infer_obb`): FP16, input 576 x 1024, class-agnostic NMS at IoU 0.7, at most 300 boxes, boxes with score > 0.1 are saved, corners re-ordered with `order_quad` so they follow the GT convention.
- **Result**: 71.8 % detection F-score on icdar2015v val (precision 83.3 %, recall 63.1 %, score > 0.5); 25.7 ms per frame on the RTX 3050.

Both detectors share the same frame loop (`infer_paddle.run`), which also records ms/frame and peak GPU memory into `runs/<det_exp>/det/timing.json` and `metrics.json meta.det_infer`.

7 Stage 2 — Language Synergy Classifier (LSC)

7.1 Idea

A scene-text recogniser is trained to read words. Asked to read a box that holds no text, a well-trained recogniser emits the end-of-sequence token immediately, i.e. it decodes the empty string. The LSC uses that behaviour as a **text / non-text classifier**: `lang_hard = 1` if anything is decoded, else 0. As a side product every kept box gets its recognised text and a visual feature tensor used later by the tracker.

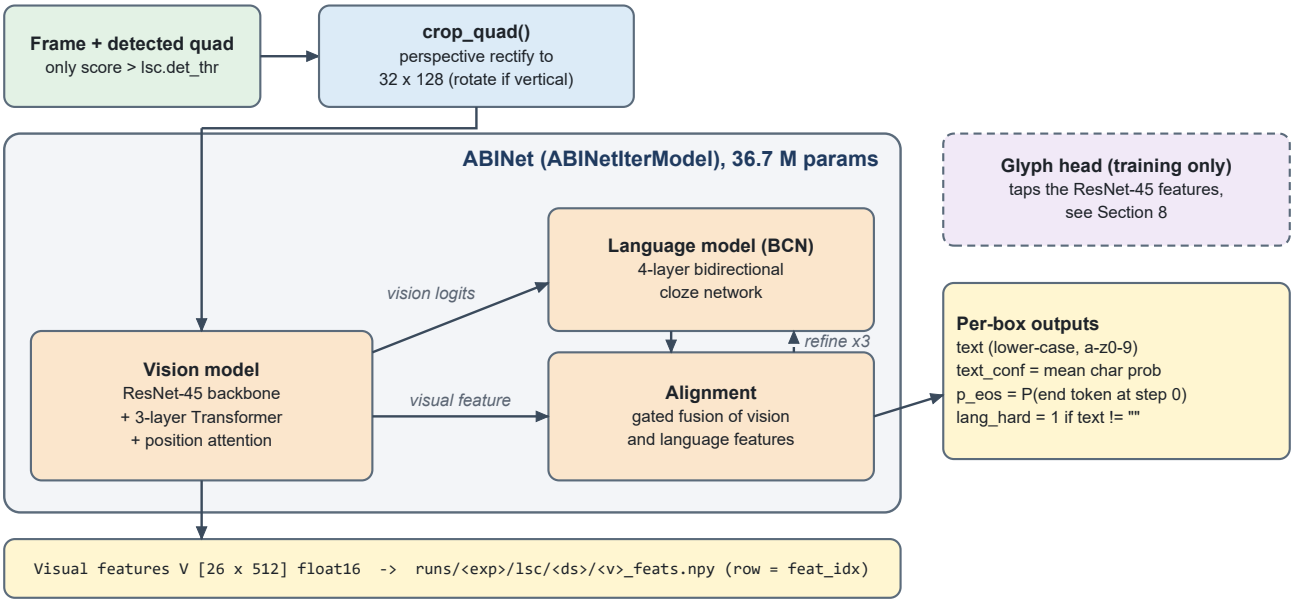


Figure 7. LSC inference for one detected box: rectification, ABINet recognition, and the outputs stored per box.

7.2 Rectifying a box (`logo.lsc.roi_rotate`)

- `order_vertices` sorts the corners clockwise around the centroid and starts at the corner whose outgoing edge points most to the right, giving top-left, top-right, bottom-right, bottom-left.
- `crop_quad` computes a perspective transform from the quad to a 32 x 128 rectangle (ABINet's input size) and warps the frame with bilinear interpolation and replicated borders. Pixel-centre conventions are handled so that an axis-aligned box crops exactly like slice-and-resize.

- If the box is vertical (height > 1.5 x width), it is warped to 128 x 32 and rotated 90° so that top-to-bottom text reads left to right.
- Degenerate quads (area < 1 px²) fall back to their axis-aligned bounding box.

7.3 The ABINet model

ABINet ("Read Like Humans", CVPR 2021) is the recogniser used as LSC. The configuration in `third_party/ABINet/configs/train_abinet.yaml` is used unchanged:

Part	Structure	Role
Vision model	ResNet-45 backbone + 3-layer Transformer encoder + position attention	Turns the 32 x 128 image into 26 reading positions x 512 features and a first character guess.
Language model (BCN)	4-layer bidirectional cloze network; gradient detached from vision	Corrects the spelling using language knowledge only (like a spell checker).
Alignment	Gated fusion of vision and language features	Final prediction; the language/alignment loop runs 3 times (iterative refinement).
Character set	36 classes: a-z and 0-9, maximum 25 characters + end token	Hence c = 26 reading positions; feature width d = 512.

`ABINetRecognizer.predict` normalises the crops with ImageNet mean/std, runs the model in batches of 64 under FP16 autocast and greedily decodes the alignment head. Per crop it returns: `text` (lower-case), `text_conf` (mean max-probability of the decoded characters, 0 for empty text), `p_eos` (probability that the very first token is the end token, i.e. that the box is *not* text) and `feat` = the vision model's attention vectors V of shape [26, 512].

7.4 The LSC stage (`logo.lsc.run_lsc`)

- Keeps boxes with detector score > `lsc.det_thr` (0.3); the rest are dropped from the pipeline here.
- Crops are buffered across frames and recognised in batches, so GPU utilisation stays high.
- Each kept box gains `text, text_conf, p_eos, lang_hard, feat_idx`. `feat_idx` is the row of the box in `<video>_feats.npy` (float16, shape [N, 26, 512]), which is written through a memory map so RAM never holds more than one batch.
- Checkpoint choice: `--ckpt`, else `runs/<exp_id>/checkpoints/best.pth`, else `lsc.ckpt` from the config, else the ABINet release weights.

7.5 Fine-tuning the LSC (`logo.lsc.train_lsc`)

The released ABINet weights were trained on synthetic data and never learned to output "nothing" for background. Fine-tuning teaches both: reading video text and rejecting non-text.

- **Dataset** (`tools/build_lsc_dataset.py` -> `data/lsc/<split>/`): *positives* are crops of readable GT boxes, labelled with their text restricted to the 36-character set; *negatives* are detector boxes with score > 0.3 whose IoU with every GT box (dont_care included) is below 0.05, labelled with the empty string. Positives and negatives are balanced 1:1. The E3-lsc run used 3,568 training and 5,026 validation crops.
- **Augmentation** (ABINet style): with probability 0.5 one geometric change (rotation ± 10°, affine with shear, or perspective); with 0.25 one degradation (Gaussian or motion blur, noise, down-up scaling, JPEG); with 0.25 colour jitter.
- **Loss**: ABINet's own `MultiLosses` = cross-entropy of the vision, language and alignment outputs over the 3 iterations. A negative is encoded as "end token at step 0", so only that one step is supervised.
- **Optimisation**: Adam, learning rate 1e-4 for epochs 1–6 and 1e-5 for 7–10, effective batch 48 (24 x 2 gradient-accumulation steps), AMP, gradient-norm clipping at 20. On a CUDA out-of-memory error the micro-batch is halved and the batch retried.
- **Model selection**: best epoch by (text/background accuracy, word accuracy) on val.

Run	Best epoch	Text/background acc. %	Word acc. % (positives)	Text recall %	Background recall %
`E3-lsc` (LSC only)	5	94.79	69.52	95.22	94.35

Run	Best epoch	Text/background acc. %	Word acc. % (positives)	Text recall %	Background recall %
`E3-lsc-glyph` (LSC + GS)	4	95.40	74.37	93.83	96.98

Validation on the LSC crop set (icdar2015v). Glyph supervision raised word accuracy by 4.85 points.

8 Glyph supervision (GS)

Glyph supervision adds a small decoder that must reconstruct a pixel mask of the characters from the recogniser's backbone features (paper Eq. 5). The extra task forces the backbone to keep stroke-level detail, which helps on blurred and tiny text. The head is used only while training; the saved model checkpoint is a plain ABINet and runs at the same speed.

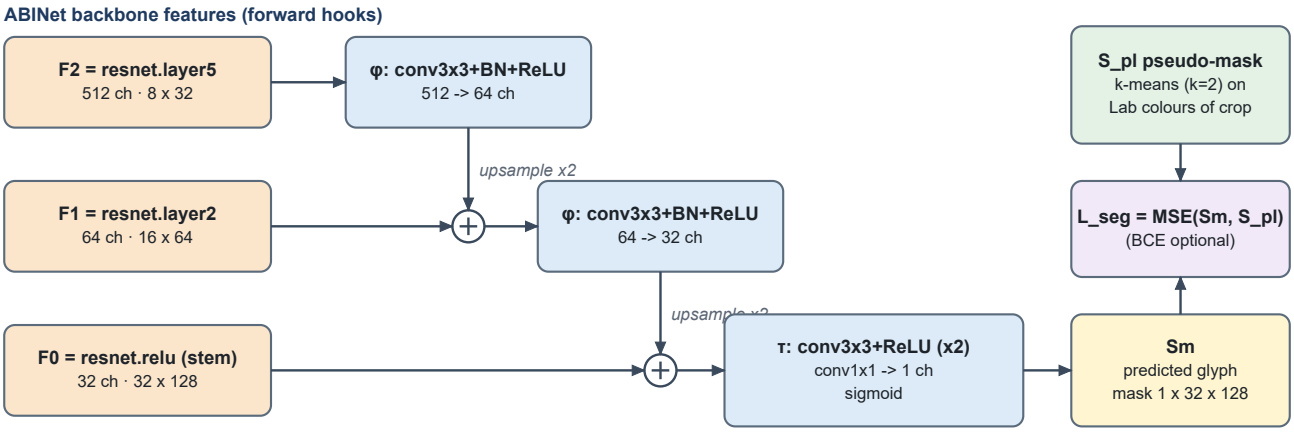


Figure 8. Glyph head (logo/glyph/branch.py). Three backbone resolutions are tapped with forward hooks and fused by a U-Net-like decoder; the predicted mask is compared with a pseudo-mask.

Where the targets come from: pseudo-masks

Real glyph masks are not annotated. `logo.glyph.pseudo_labels.kmeans_mask` creates them without supervision: the pixels of each positive crop are converted to Lab colour space and clustered into two groups with seeded k-means++ (best of 3 runs). The cluster that touches the crop border less is called "text" (this works for both dark-on-light and light-on-dark text); a 3 x 3 closing fills stroke gaps and a cross-shaped opening removes specks. `tools/make_glyph_masks.py` writes them to `data/lsc/<split>/masks/`.

How it is trained

With `glyph.enabled: true` the training loss becomes $\mathbf{L = L_{rec} + \lambda \cdot L_{seg}}$ with λ = `glyph.weight` = 1 and L_{seg} = mean-squared error between the sigmoid mask and the pseudo-mask (resized to 32 x 128). Crops without a mask are excluded from L_{seg}. `GlyphLSCDataset` applies the same random geometric warp to image and mask, and draws exactly the same random numbers as the plain dataset, so runs with and without GS see identical training crops — a fair ablation.

9 Stage 3 — Score fusion and the C2 extension

Fusion combines the detector's visual confidence with the LSC's language evidence (`logo.scoring.fusion`):

> **score_fused = (1 − w) · det_score + w · lang**
> **hard** (LOGO): lang = lang_hard (0 or 1); with w = 0.5 this is the paper's average of the two scores.
> **soft** (C2, this project): lang = 1 − p_eos (probability that the first decoded token is *not* the end token), or text_conf when `fusion.soft_source: text_conf`.

Hard fusion gives every box that yields at least one character the same +0.5 bonus, however unsure the recogniser was. Soft fusion (C2) passes the recogniser's actual confidence through, so borderline boxes move

only a little. `w = 0` switches the language signal off; the Baseline experiment uses this. `logo.scoring.run_fuse` copies every lsc object and adds `score_fused`; `feat_idx` keeps pointing into the LSC experiment's feature file (features are not copied).

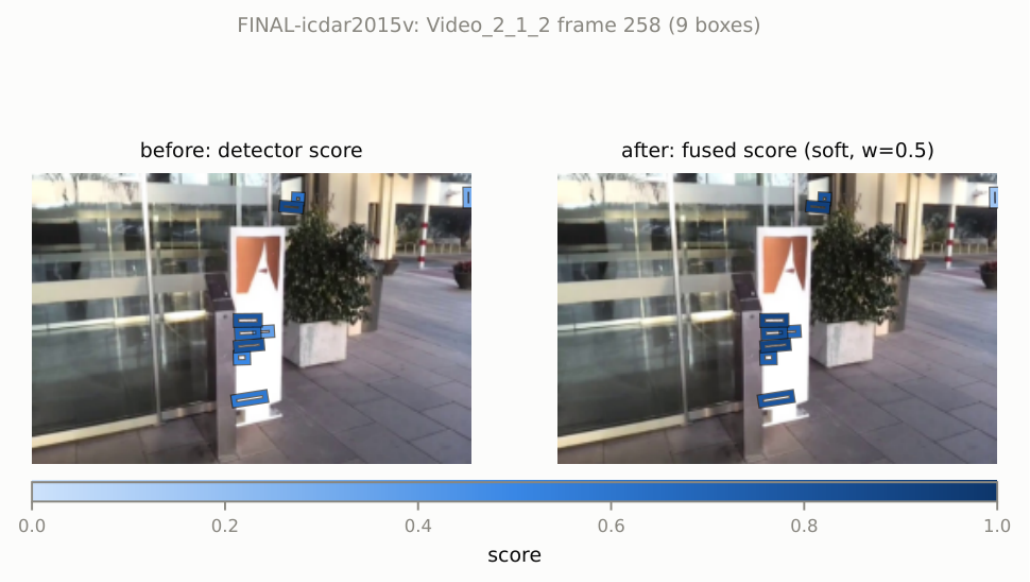
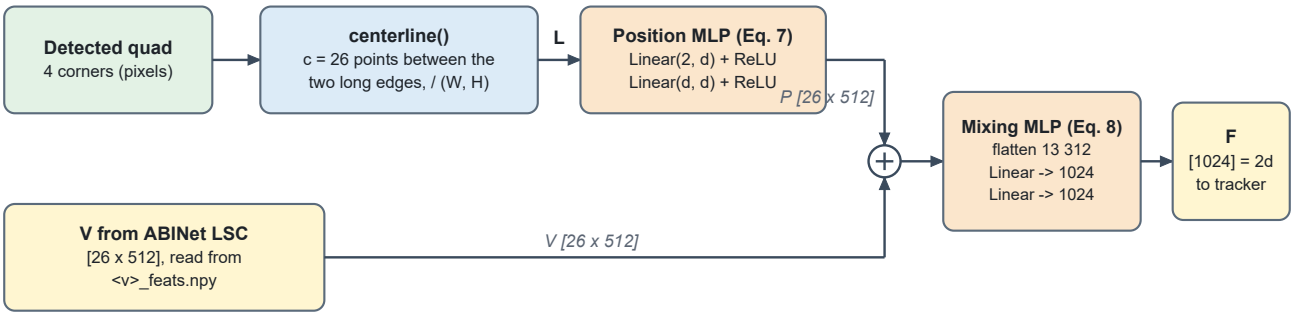


Figure 9. One frame before and after soft re-scoring (paper/figs/rescore_*.pdf): box colour is the score, light = 0, dark = 1.

10 Visual-Position Mixed Module (VPMM)

Linking boxes across frames needs a feature that says "this is the same text as before". VPMM (paper Eq. 7–8, `logo/vpmm/`) builds it from two sources: *what the text looks like* (the LSC features V) and *where it is and how it is oriented* (its centerline).



d = 512 (ABINet feature width), c = 26 reading positions; VPMM has 14.9 M parameters.

Figure 10. VPMM. The centerline position embedding P is added to the LSC visual features V position by position, flattened and mixed by an MLP into one 1024-d vector per box.

- **Centerline** (`vpmm/geometry.centerline`): the two long edges of the quad are found (ties broken deterministically so the result does not depend on vertex order), c = 26 equally spaced points are sampled on each, oriented left-to-right, and the midpoints form the centerline. Coordinates are divided by image width and height, giving L in [0, 1] of shape [26, 2].
- **Eq. 7**: P = ReLU(W_2 ReLU(W_1 L)), shape [26, 512]. Each reading position gets a learned embedding of its location.
- **Eq. 8**: F = MLP(flatten(P + V)), 26 x 512 = 13,312 -> 1,024 -> 1,024 with ReLU. Because position i of the centerline lines up with reading position i of ABINet, the sum mixes appearance and geometry per character slot.

VPMM has no loss of its own: it is trained jointly with the tracker that consumes F (Section 11.3).

11 Stage 4 — Tracking

`logo.tracking.run_track` turns fused detections into trajectories. `tracker.type` selects one of three trackers; all write the same jsonl rows `{"frame", "track_id", "quad", "score", "text"}`.

tracker.type	Association cue	Learned parts	Used by
`simple`	Box IoU with constant-velocity motion prediction (ByteTrack-style, two passes)	none	Baseline, +LSC hard, +GS rows
`embed`	IoU + cosine distance of a learned appearance embedding	VPMM + projection head, SupCon loss	fallback (P21F); not in the final ablation
`lst`	LST-Matcher Transformers on VPMM features + IoU	VPMM + LSTAdapter (42.2 M params)	+VPMM/LST rows, extensions, FINAL

11.1 The IoU tracker (`tracker.type: simple`)

1. Frames are processed in order. Tracks unseen for more than `max_age` = 15 frames are deleted.
2. Each live track predicts its quad by moving the last quad with its centre velocity.
3. **Pass 1**: detections with score_fused > 0.6 are matched to all live tracks with the Hungarian algorithm on quad IoU (computed exactly with shapely polygons; pairs below IoU 0.3 are rejected).
4. **Pass 2**: tracks still unmatched are matched the same way to the low-score detections (0.3 < score <= 0.6). This keeps tracks alive through brief drops in confidence without letting weak boxes start tracks.
5. Unmatched high-score detections start new tracks; other unmatched detections are discarded.
6. The text of every output box is the text of the most confident detection among the track's last 10 matches.

11.2 The embedding tracker (`tracker.type: embed`)

Same two-pass algorithm, but pass 1 uses the cost $(1 - \lambda)(1 - \text{IoU}) + \lambda \cdot$ cosine distance between the track's embedding (exponential moving average, momentum 0.9) and the detection's embedding, $\lambda = 0.5$. The embedding is VPMM followed by a 2-layer head to 128 dimensions, trained by `logo.tracking.train_embed` with a supervised contrastive (SupCon) loss on 6-frame clips: boxes of the same GT track are positives, everything else negatives.

11.3 The LST-Matcher tracker (`tracker.type: lst`)

This is LOGO's tracker. GoMatching's original LSTMatcher expects detectron2 objects and DeepSolo query features; `LSTAdapter` (`logo/tracking/lst_adapter.py`) re-hosts its association logic on plain tensors and feeds it VPMM vectors instead (a recorded deviation from the paper). Its parts keep GoMatching's parameter names:

- `asso_head`: 2 fully-connected layers (2d = 1024 -> 1024) producing the association embedding.
- `short_term_matcher` and `long_term_matcher`: GoMatching Transformers (1 encoder + 1 decoder layer, 8 heads, width 1024, no positional embedding, as in every GoMatching config).
- `local_asso_predictor` / `asso_predictor`: association logits = q_proj(query) · k_proj(key)^T; a softmax with an extra "background" column gives the probability that box i matches box j or nothing.

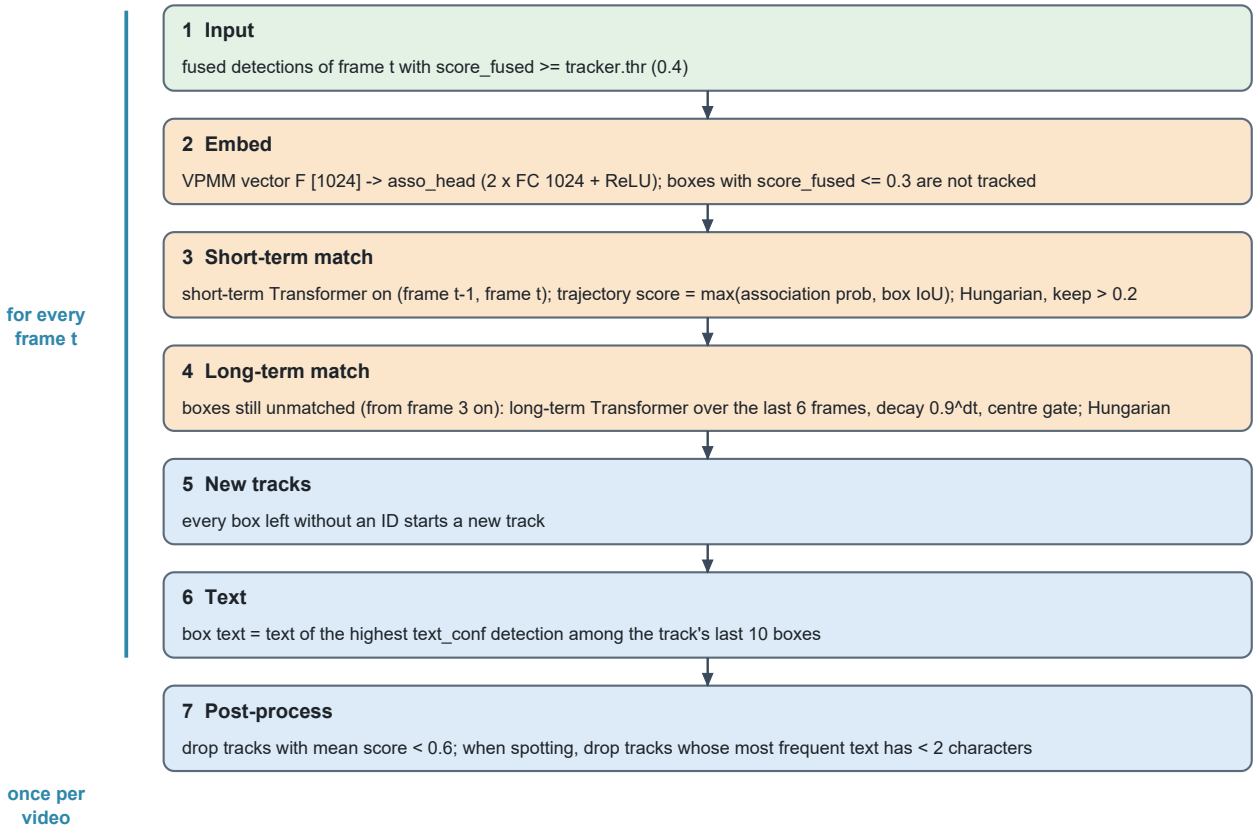


Figure 11. Online association by LSTAdapter.associate (steps 1–6, per frame) and the post-processing in run_track (step 7, per video). Numbers are the values in configs/FINAL.yaml and LST_DEFAULTS.

Training (`logo.tracking.train_lst`)

- **Cache** (`tools/build_lst_cache.py` -> `data/cache/<ds>/<video>.npz`): fused boxes with score > 0.3, their LSC feature rows, and a GT track ID from a one-to-one IoU >= 0.5 match (-1 if unmatched). Features are memory-mapped straight from the npz.
- **Samples**: clips of 6 consecutive frames of one video (`LSTCacheDataset`).
- **Loss**: DETR-style association cross-entropy. Long-term: every box against every frame of the clip; short-term: every pair of consecutive frames; total = $0.5 \cdot L_{long} + 0.5 \cdot L_{short}$. Unmatched boxes are trained to pick “background”.
- **Optimisation**: AdamW, lr 5e-5, weight decay 1e-4, linear warm-up over 1,000 iterations then cosine decay, AMP, gradient clipping 0.1; on CUDA out-of-memory the clip length is reduced by one. VPMM and the adapter are trained jointly.
- **Validation**: every 1,000 iterations the val caches are tracked and scored with the evaluator; best = (IDF1, MOTA). `last.pth` holds optimiser, scheduler, scaler, RNG and data position for `--resume`.

The run `E5-lst` completed 9,000 of the planned 30,000 iterations; `best.pth` is from iteration 1,000. Limitations of this checkpoint are discussed in Section 19.2.

Inference post-processing (paper)

Before association, detections with score_fused < `tracker.thr` (0.4 for ICDAR, 0.3 for DSText) are dropped. After the whole video, trajectories with mean score < `traj_thr` = 0.6 are removed and, when spotting, so are trajectories whose most frequent text has fewer than two characters.

12 Stage 5 — Track-level text voting (C3 extension)

The tracker labels each box with the most confident recent reading, so the text of one track can flicker between spellings (“exit”, “exi”, “exlt”). Spotting metrics demand an exact match, so flicker costs points. Voting (`logo.voting.track_vote`) gives each track one text:

1. For every box of a track, look up its `text_conf` and language score `lang` in the fused file (matched by frame and quad).
2. Normalise each box text (lower-case, keep a-z0-9) and add the weight text_conf · lang to its group.
3. The group with the highest total weight wins (ties: higher mean text_conf, then alphabetical). The empty string wins only if no box read anything.
4. The winning group’s most heavily weighted original spelling is written to *every* box of the track.
5. Tracks with fewer than `voting.min_len` = 3 boxes or mean lang < `voting.tau` = 0.5 are deleted.

If `voting.enabled` is false the stage copies the track files unchanged, so evaluation always reads `vote/`.

13 Stage 6 — Evaluation

The evaluator (`logo/eval/`, driven by `tools/evaluate.py`) is used by the pipeline, the training loops and every analysis tool, so all numbers are computed the same way.

13.1 Matching rules (`logo.eval.match`)

- IoU is the exact polygon IoU (shapely), with an axis-aligned pre-filter for speed. A match needs IoU >= 0.5 (`eval.iou`).
- dont_care GT boxes are removed. A prediction lying more than 50 % inside a dont_care box is ignored (neither true nor false positive).
- Frames that only have predictions are evaluated too (all their boxes are false positives).
- Spotting mode additionally requires the normalised texts (lower-case, a-z0-9) to be equal, and treats GT boxes with no alphanumeric character as dont_care.

13.2 Metrics

Metric	Definition	Reads
Precision, Recall, F	Greedy one-to-one matching per frame, highest IoU first; P = TP/(TP+FP), R = TP/(TP+FN), F = harmonic mean. In the pipeline only boxes with score_fused > `eval.det_score_thr` (0.5) are counted.	`fuse/`
MOTA	1 − (misses + false positives + ID switches) / number of GT boxes. Can be negative.	`vote/`
MOTP	Mean IoU of matched pairs (reported as 100 · (1 − mean distance)).	`vote/`
IDF1	2 · IDTP / (2 · IDTP + IDFP + IDFN): how well predicted IDs agree with GT identities over the whole video.	`vote/`
MT / ML	Share of GT tracks covered in >= 80 % (mostly tracked) or <= 20 % (mostly lost) of their visible frames.	`vote/`

CLEAR-MOT accumulation uses py-motmetrics with distance 1 − IoU (impossible pairs set to NaN). Totals are pooled over all videos of the split; per-video counts are kept in `metrics.json["per_video"]` so that `tools/bootstrap_ci.py` can compute paired bootstrap confidence intervals by re-sampling videos.

> **Why spotting MOTA can be negative.** In spotting mode a box whose position is right but whose text is wrong counts as one miss *and* one false positive. When the recogniser often misreads, the error count exceeds the number of GT boxes and MOTA drops below zero, even though the boxes themselves are tracked well.

14 File formats and run layout

14.1 Stage files

Stage	File	Content (per box unless stated)
det	`runs/<det_exp>/det/<ds>/<v>.json`	`quad[8], score`; boxes sorted by score, score > save_thr (0.1)
lsc	`runs/<lsc_exp>/lsc/<ds>/<v>.json`	det keys + `text, text_conf, p_eos, lang_hard, feat_idx`; only boxes with score > lsc.det_thr
lsc	`runs/<lsc_exp>/lsc/<ds>/<v>_feats.npy`	float16 [N, 26, 512]; row feat_idx = box
fuse	`runs/<exp>/fuse/<ds>/<v>.json`	lsc keys + `score_fused`
track	`runs/<exp>/track/<ds>/<v>.jsonl`	one line per box: `frame, track_id, quad, score (= score_fused), text`
vote	`runs/<exp>/vote/<ds>/<v>.jsonl`	same as track; text voted, weak tracks removed
eval	`runs/<exp>/metrics.json`	`{"det": {...}, "track": {...}, "spot": {...}, "per_video": {...}, "meta": {...}}`
cache	`data/cache/<ds>/<v>.npz`	LST training cache (frame_ids, quads, feats, score_fused, track_ids, frames, frame_offsets, img_wh)

14.2 Anatomy of a run folder

```
runs/A-all_yolo/
  config.yaml          exact config used (written once)
  log.txt              appended log of every command run for this exp_id
  metrics.json         det / track / spot / per_video results
                       meta.{pipeline_<ds>_<split>, eval_<ds>_<split>, fuse_..., track_..., efficiency_...}
  fuse/icdar2015v/<video>.json
  track/icdar2015v/<video>.jsonl
  vote/icdar2015v/<video>.jsonl
  checkpoints/        (training runs only) best.pth, last.pth, epoch_XX.pth
```

Training runs (`E3-lsc`, `E5-lst`, ...) also write their training history to `metrics.json` under `meta.lsc_train` / `meta.lst_train`. A `_smoke` suffix marks a short test run of a training command.

15 Configuration system

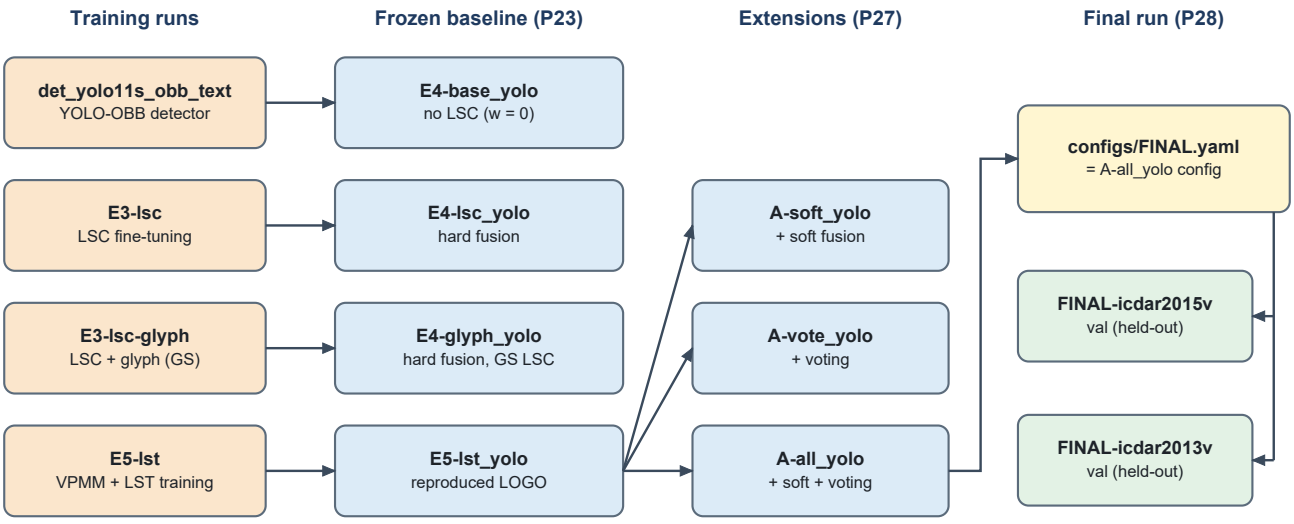
Every command takes one YAML file. `configs/base.yaml` shows every pipeline key; experiment configs override a few. Missing tracker/LSC keys fall back to defaults defined in code (`DEFAULTS`, `LST_DEFAULTS`, `TRAIN_DEFAULTS`).

Key	Default	Meaning
`exp_id, dataset, split, seed`	base, icdar2015v, val, 42	Run name, dataset, split, random seed
`pipeline.det_exp / lsc_exp`	= exp_id	Experiments whose det / lsc outputs are re-used
`detector.*`	—	weights, ppdet config (Plan A), input_hw, batch_size, save_thr, half, nms_iou, obb_data, train
`lsc.ckpt / det_thr / iou_neg`	null / 0.3 / 0.05	Recogniser weights; min det score for LSC; negative-crop IoU limit
`lsc.infer.batch_size / amp`	64 / true	LSC inference batching and FP16
`lsc.train.*`	see 7.5	LSC fine-tuning hyper-parameters
`glyph.enabled / weight / loss`	false / 1.0 / mse	Glyph supervision switch, λ, loss type
`fusion.mode / w / soft_source`	hard / 0.5 / p_eos	Fusion formula (Section 9)
`tracker.type`	simple	simple | embed | lst
`tracker.thr / traj_thr / ckpt`	0.4 / 0.6 / runs//checkpoints/best.pth	LST input threshold, trajectory filter, weights
`tracker.train.*`	see 11.3	Tracker training hyper-parameters

Key	Default	Meaning
`voting.enabled / tau / min_len`	false / 0.5 / 3	C3 voting (Section 12)
`eval.iou / spotting / det_score_thr`	0.5 / true / 0.5	Evaluation settings

15.1 Experiment family

Config	Purpose
`det_yolo11s_obb_text.yaml`	Plan B detector: data prep, training, inference.
`det_ppyoloe_r_s.yaml, det_ppyoloe_r_text_s.yaml`	Plan A detector (DOTA weights and text fine-tuned).
`E3-lsc.yaml, E3-lsc-glyph.yaml`	LSC fine-tuning without / with glyph supervision.
`E4-base.yaml, E4-lsc.yaml, E4-glyph.yaml`	Pipeline rows: no LSC (w = 0), hard LSC, hard LSC with GS weights.
`E5-lst.yaml, E5-embed.yaml`	LST-Matcher and embedding tracker: training + pipeline.
`freeze_baseline.yaml, ablation.yaml`	Row plans for the frozen baseline (P23) and the full ablation (P27).
`FINAL.yaml`	The final system (= A-all_yolo): YOLO-OBB, E3-lsc LSC, soft fusion, LST tracker, voting.
`paper.yaml, reported_numbers.yaml`	Inputs of the table/figure generators; published numbers (still empty).



all P23/P27 rows read det boxes from det_yolo11s_obb_text and LSC outputs from E4-lsc_yolo where shared

Figure 12. How experiments build on each other. Training runs produce weights; the frozen baseline adds one component per row; the extension rows add C2/C3; the final config is A-all_yolo.

16 Running the project

16.1 Setup

```
uv venv --python 3.11 .venv
.venv\Scripts\Activate.ps1
uv pip install torch==2.1.2 torchvision==0.16.2 --index-url https://download.pytorch.org/whl/cu121
uv pip install -e ".[dev]"
python tools/check_env.py          # 0 = ok, 1 = missing package, 2 = no CUDA
pytest -q                          # 250 tests
```

16.2 End-to-end recipe

The order below rebuilds everything from raw data. Each command is resumable.

```
# 1  data
python tools/inventory.py --root data/raw
python tools/make_splits.py --ds icdar2013v icdar2015v dstext
python tools/extract_frames.py --ds icdar2015v --split train val
python tools/convert_annotations.py --ds icdar2015v
python tools/dataset_stats.py
# 2  detector (Plan B)
python -m logo.detector.prepare_obb_data --config configs/det_yolo11s_obb_text.yaml
python tools/train_obb.py --config configs/det_yolo11s_obb_text.yaml
python -m logo.detector.infer_obb --config configs/det_yolo11s_obb_text.yaml --split train
python -m logo.detector.infer_obb --config configs/det_yolo11s_obb_text.yaml --split val
# 3  LSC (+ glyph supervision)
python tools/build_lsc_dataset.py --config configs/E3-lsc.yaml --det_exp det_yolo11s_obb_text --splits train val
python -m logo.lsc.train_lsc --config configs/E3-lsc.yaml
python tools/make_glyph_masks.py --config configs/E3-lsc-glyph.yaml --splits train
python -m logo.lsc.train_lsc --config configs/E3-lsc-glyph.yaml
# 4  tracker training data and LST training
python -m logo.lsc.run_lsc --config configs/E4-lsc.yaml --det_exp det_yolo11s_obb_text --splits train val
python -m logo.scoring.run_fuse --config configs/E4-lsc.yaml --splits train val
python tools/build_lst_cache.py --config configs/E4-lsc.yaml --splits train val
python -m logo.tracking.train_lst --config configs/E5-lst.yaml
# 5  experiments, final evaluation, paper material
python tools/freeze_baseline.py
python tools/run_ablation.py
python scripts/final_eval.py --config runs/A-all_yolo/config.yaml --final
python tools/bootstrap_ci.py E5-lst_yolo A-all_yolo
python tools/make_tables.py ; python tools/make_figures.py
```

To run a single experiment through the pipeline: `python scripts/run_pipeline.py --config configs/E4-lsc.yaml --stage lsc,fuse,track,vote,eval --det_exp det_yolo11s_obb_text`. Stages between the first and last requested stage always run when an earlier one ran, so a stage never reads stale input.

16.3 Demo

```
python scripts/demo.py --video in.mp4 --out out.mp4                # configs/FINAL.yaml, CUDA if available
python scripts/demo.py --video in.mp4 --out out.mp4 --device cpu
python scripts/demo.py --gradio                                    # upload page on http://127.0.0.1:7860
```

The demo builds a throw-away project tree in a temporary folder (`LOGO_ROOT`), extracts the frames, runs the *same* stage functions as the pipeline, draws quads with `#track_id text` labels into an mp4 and prints the number of tracks, the longest track and the frames per second. If the config names a PaddleDetection config, the detector runs as a subprocess in the Paddle environment.



Figure 13. Output of the final system on ICDAR2015 Video_2_1_2: each box is labelled with its track ID and voted text (paper/figs/overlay_*.pdf).

17 Tool catalogue

Every tool is a stand-alone script with a docstring that lists its inputs, outputs and usage. They fall into six groups.

Group	Tools	Purpose
Data audit & preparation	`inventory`, `audit_dstext_annotations`, `inspect_dstext_xml`, `dataset_audit`, `convert_annotations`, `extract_frames`, `make_splits`, `dataset_stats`, `viz_gt`	Check raw data, build GT json, frames and splits, data statistics, draw GT.
Detector	`gt_to_dota`, `train_detector`, `train_obb`, `export_all_dets`	Build detector training sets, train both detectors, export detections for all splits.
LSC & glyph	`build_lsc_dataset`, `make_glyph_masks`, `eval_recognition`, `compare_glyph`, `crop_grid`, `mask_grid`	Crop datasets, pseudo-masks, recognition accuracy, GS comparison on noisy crops, visual checks.
Tracker	`build_lst_cache`	Training cache for LST / embedding trackers.
Evaluation & experiments	`evaluate`, `baseline_report`, `freeze_baseline`, `run_ablation`, `compare_runs`, `sweep_fusion`, `sweep_threshold`, `sweep_voting`, `bootstrap_ci`, `profile_efficiency`, `error_analysis`	Scoring, frozen baseline, ablation, parameter sweeps, significance tests, speed/memory profiling, error categories.
Reporting	`make_tables`, `make_figures`, `rescore_panel`	LaTeX + Markdown tables and PDF figures generated only from run outputs (no hand-typed numbers).
Environment	`check_env`	Verify Python, packages and CUDA.

18 Experiments and results

All numbers are percentages on the icdar2015v validation split (5 videos) unless stated, IoU 0.5, Plan B detector. They are copied from `docs/tables/`, which `tools/make_tables.py` generates from `metrics.json` files.

18.1 Ablation

Row	Run	Det F	Track MOTA	Track MOTP	Track IDF1	Spot MOTA	Spot MOTP	Spot IDF1
Baseline (no LSC, IoU tracker)	`E4-base_yolo`	71.80	46.01	**74.74**	**57.80**	-6.20	**77.50**	33.90
+ LSC hard fusion	`E4-lsc_yolo`	**73.51**	43.83	74.58	56.44	-9.84	77.33	33.38
+ GS	`E4-glyph_yolo`	73.30	43.51	74.56	56.32	**-4.24**	76.80	**36.08**
+ VPMM / LST (= LOGO)	`E5-lst_yolo`	**73.51**	45.52	74.69	51.95	-10.47	77.43	30.31
+ soft fusion (C2)	`A-soft_yolo`	73.35	45.13	74.64	52.76	-10.77	77.40	30.97
+ voting (C3)	`A-vote_yolo`	**73.51**	**46.23**	74.62	52.26	-13.89	76.83	30.81
all (C2 + C3) = FINAL	`A-all_yolo`	73.35	45.80	74.58	53.06	-13.13	76.94	32.15

Reading the table.

- The LSC improves detection (F +1.7 points) by pushing non-text boxes below the threshold, but with the IoU tracker it lowers tracking MOTA by 2.2 points.
- Glyph supervision gives the best spotting results (spot IDF1 36.08, spot MOTA -4.24), consistent with its higher word accuracy (Section 7.5).
- Replacing the IoU tracker with VPMM + LST raises MOTA back to 45.5 but lowers IDF1 by 4.5 points relative to +GS. See Section 19.2 for the most likely cause.
- C2 + C3 on top of LST add +0.28 MOTA and +1.11 IDF1. The paired bootstrap 95 % interval of the IDF1 gain is [-0.09, 3.42], so the gain is not statistically significant on five validation videos.

18.2 Final held-out evaluation

Dataset (split)	Det P / R / F	Track MOTA	MOTP	IDF1	MT	ML	Spot MOTA	Spot IDF1
ICDAR2015 Video (val)	81.11 / 66.95 / 73.35	45.80	74.58	53.06	40.97	39.65	-13.13	32.15
ICDAR2013 Video (val)	84.27 / 85.11 / 84.68	66.18	77.44	61.54	47.71	43.71	19.93	49.57

`scripts/final_eval.py` ran `FINAL.yaml` unchanged except for dataset/split/exp_id, after checking that no hyper-parameter differs, and stored the git commit, config hash and GPU in `meta.final`. The published LOGO numbers have not been entered in `configs/reported_numbers.yaml` yet, so the comparison tables show “-” for the paper.

18.3 Efficiency (batch 1, RTX 3050 6 GB, 200 frames)

Run	Det	LSC	Fuse	VPMM	Track	Total ms/frame	FPS	Params (M)	Peak VRAM MB
`E4-base_yolo`	25.7	36.5	0.0	-	0.6	62.7	15.95	46.4	303.5
`E4-lsc_yolo`	25.7	55.6	0.0	-	0.1	81.4	12.29	46.4	304.5
`E4-glyph_yolo`	25.7	26.1	0.0	-	0.2	51.9	19.26	46.4	304.5
`E5-lst_yolo`	25.7	26.3	0.0	2.6	3.6	58.2	17.20	88.7	304.5
`FINAL-icdar2015v`	25.7	66.6	0.0	7.2	9.6	109.0	9.17	88.7	303.4

The LSC dominates the cost; its time varies between runs mainly with the number of boxes above the 0.3 threshold. Parameter counts: detector 9.70 M, ABINet 36.74 M, VPMM 14.95 M, LST adapter 27.29 M. Peak GPU memory stays near 300 MB at batch 1, far below the 6 GB budget.

18.4 Error analysis

`tools/error_analysis.py` classifies every error of a run into missed GT, false positive, wrong text, ID switch and fragmentation, split by text height (< 20, 20–50, > 50 px), and draws example galleries. For the final system on icdar2015v val it counts 2,607 missed GT boxes, 2,266 matches with wrong text, 950 false positives, 353 ID switches and 202 fragmentations. Missed boxes and misreadings dominate, most errors fall in the 20–50 px band, and one video (Video_37_2_3) accounts for about 59 % of all errors.

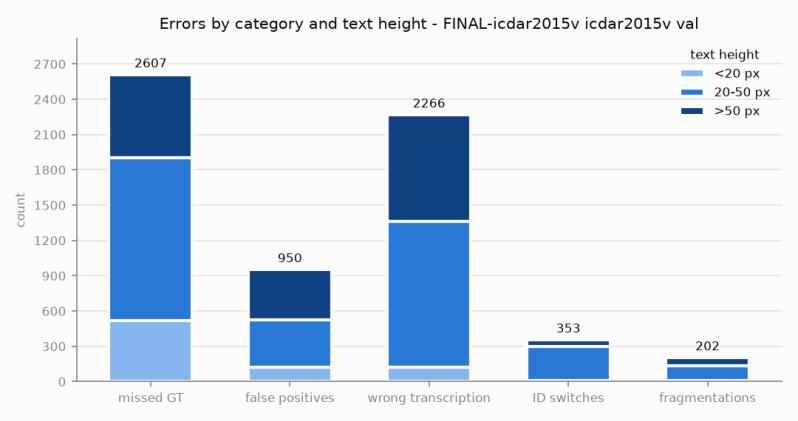


Figure 14. Error counts per category and text-height bin for FINAL-icdar2015v (docs/figs/errors/errors_bar.png).

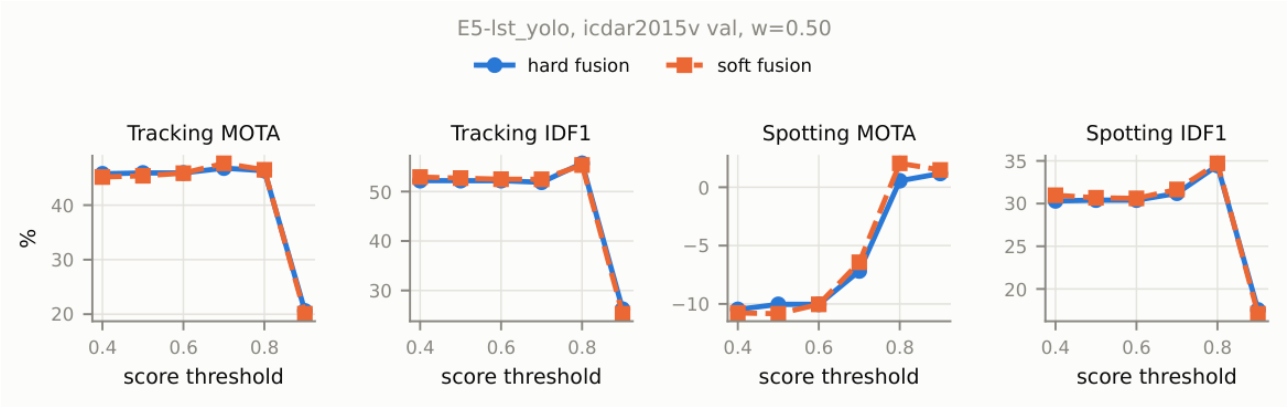


Figure 15. MOTA and IDF1 against the score threshold for hard and soft fusion (tools/sweep_fusion.py, paper/figs/threshold_E5-lst_yolo_icdar2015v_val.pdf).

19 Design decisions, deviations and limitations

19.1 Decisions and deviations from the paper

Decision	Reason	Effect
Detector: YOLO11s-OBB instead of the paper's detector; input 1024 x 576	6 GB GPU; PP-YOLOE-R fine-tuning failed	Different detector than LOGO; results not directly comparable
LST-Matcher fed with VPMM features [N, 1024] instead of DeepSolo query features	No DeepSolo / detectron2; keep third_party untouched	Recorded deviation (P19)
Rescoring head, positional embeddings, NMS and min-track-length of GoMatching dropped	Replaced by fusion, NO_POS_EMB in all GoMatching configs, upstream NMS, voting.min_len	Smaller adapter, same association maths
Atomic json writes, append-only logs	Interrupted runs must not leave half files	None on results
Validation split used as held-out set	Test labels unavailable (ICDAR2015, DSText)	Final numbers are on val; optimistic for icdar2013v (also used for tuning)
Glyph head is training-only, saved separately	Keep checkpoints loadable by plain ABINet	No inference cost

19.2 Known limitations and open items

- **LST checkpoint quality.** `E5-lst` was trained and validated on caches built from Plan A (PP-YOLOE-R) detections, whose validation recall was near zero. The selected checkpoint (iteration 1,000) scored only 0.08 % IDF1 on that cache, and training stopped at 9,000 of 30,000 iterations. The LST rows on YOLO boxes still track reasonably, plausibly because the trajectory score takes the maximum of the learned association and plain box IoU (`with_iou: true`). Re-building the cache from `det_yolo11s_obb_text` boxes and re-training is the most promising improvement.
- **Recognition is the bottleneck for spotting.** Spot MOTA is negative on icdar2015v: wrong readings count twice (Section 13).
- **Small validation set.** icdar2015v val has 5 videos; bootstrap intervals are wide and none of the differences tested is significant.
- **Paper numbers missing.** `configs/reported_numbers.yaml` and the `paper_table_v` block of `freeze_baseline.yaml` are still null.
- **ICDAR2013 held-out record.** `docs/decisions.md` says test labels are unavailable for icdar2013v, but the raw-data inventory lists 15 test GT XML files for it. Converting them (`tools/convert_annotations.py --final`) would allow a true test-split result.
- **DSText** is supported by every stage but has no end-to-end result yet.
- **Temporary config edits.** `configs/E3-lsc.yaml` currently has batch size 4, val batch 2 and 0 workers with “change back” comments; the recorded E3-lsc run used the values documented in Section 7.5.
- **Web layer.** `ARCHITECTURE.md` specifies a FastAPI + React service that has not been implemented.

20 Testing

The `tests/` folder holds 52 pytest files with 250 tests; all pass (1 min 45 s on the development laptop). Tests use small synthetic inputs (tiny images, hand-made quads, fake recognisers and predictors), so most run without a GPU or datasets.

Area	Example test files	What is checked
Geometry & I/O	`test_geometry, test_io, test_paths_loaders, test_schema_splits`	quad ordering, IoU, atomic writes, path rules, rule 5
Data	`test_parsers, test_convert_gt, test_frames, test_make_splits, test_inventory`	XML parsing quirks, GT schema, frame naming, split disjointness
Detector	`test_infer_obb, test_infer_paddle, test_prepare_obb_data, test_gt_to_dota`	row conversion, thresholds, label files
LSC & glyph	`test_roi, test_abinet_wrapper, test_run_lsc, test_train_lsc, test_glyph_branch, test_glyph_masks`	crop geometry, decoding, feature files, losses, mask generation
Fusion, VPMM, tracking	`test_fusion, test_vpmm, test_simple_tracker, test_embed_tracker, test_lst_adapter, test_train_lst`	fusion maths, centerline, association, training step
Voting & evaluation	`test_track_vote, test_eval, test_bootstrap_ci, test_error_analysis`	voting rules, metric values on hand-built cases
Drivers & reports	`test_run_pipeline, test_final_eval, test_run_ablation, test_demo, test_make_tables, test_make_figures`	stage planning, skip logic, config freezing, generated tables

Appendix A Model inventory

Model	Source	Params	Input	Trained by
YOLO11s-OBB	Ultralytics, DOTA-pretrained	9.70 M	1024 x 576 frame	`tools/train_obb.py`
PP-YOLOE-R-s (Plan A)	PaddleDetection, DOTA-pretrained	—	1024 x 576 frame	`tools/train_detector.py`
ABINet (LSC)	third_party/ABINet release weights	36.74 M	32 x 128 crop	`logo.lsc.train_lsc`
Glyph head	this project (Eq. 5)	small, training only	ResNet-45 features	`logo.lsc.train_lsc` (glyph.enabled)
VPMM	this project (Eq. 7–8)	14.95 M	quad + V [26 x 512]	`train_lst` / `train_embed`
LSTAdapter	GoMatching Transformer + heads	27.29 M	VPMM vectors	`logo.tracking.train_lst`
Embedding head	this project	small	VPMM vectors	`logo.tracking.train_embed`

Appendix B References

- LOGO: Video Text Spotting with Language Collaboration and Glyph Perception Model. arXiv:2405.19194.
- Fang et al. Read Like Humans: Autonomous, Bidirectional and Iterative Language Modeling for Scene Text Recognition (ABINet). CVPR 2021.
- He et al. GoMatching: A Simple Baseline for Video Text Spotting via Long and Short Term Matching. 2024.
- Ultralytics YOLO11 (oriented bounding boxes); PaddleDetection PP-YOLOE-R.
- Zhang et al. ByteTrack: Multi-Object Tracking by Associating Every Detection Box. ECCV 2022.
- Khosla et al. Supervised Contrastive Learning. NeurIPS 2020.
- Bernardin and Stiefelhagen. Evaluating Multiple Object Tracking Performance: the CLEAR MOT Metrics. 2008; Ristani et al. Performance Measures and a Data Set for Multi-Target, Multi-Camera Tracking (IDF1). 2016.